

Algebraic Geometry I: Final

Numerical Algebraic Geometry

Matej Gajdos (r1085570)

January 25, 2026

Contents

1	Introduction	1
2	Preliminaries and Notation	2
3	Finding Isolated Roots Using Homotopic Continuation	4
3.1	General Structure of Homotopic Continuation	6
3.2	Homotopies	8
3.3	Path Tracking	9
3.4	Intermezzo: Problematic Points in Path Tracking	10
3.5	Endgames	11
3.6	Parametric Continuation	14
3.7	Recasting Nonsquare Systems to a Square Form	15
3.8	Homotopic Continuation in Bertini	16
4	Applications	18
4.1	Polynomial Optimisation and Optimal Control	18
4.1.1	Optimisation and Polynomial Optimisation	18
4.1.2	Optimal Control	20
4.1.3	Experiment	21
5	Conclusion	24

1 Introduction

Numerical algebraic geometry (NAG) is a relatively young discipline aiming to describe solution sets of polynomial systems using numerical methods. This is in contrast

with symbolic algebraic geometric methods (utilising e.g. Gröbner bases), which tries to represent objects exactly and manipulates polynomials as one of its fundamental objects [1, 2].

Numerical algebraic geometry pursues a different approach working with precision, floating point arithmetic (FPA) and more general numerical methods, which are typically formulated not only for polynomial problems [1, 2, 3]. However, the specialisation of NAG to polynomials leads to strong results about robustness, stability and convergence of those numerical methods. Even though the numerical approach deals with rounding and inexactness, the results can be made very trustworthy, with the additional advantage that these methods tend to be faster and scale better with size of problems than their exact symbolic counterparts (see e.g. Part I, Chapter 6.6. in [3] for comparison of those methods).

The main aim of this short expository paper is to examine how the cornerstone of NAG, the *homotopic continuation*, works. This method primarily concerns with finding isolated solutions of polynomial systems where the number of variables matches the number of equations. However, even the algorithms studying non-isolated solutions utilise homotopic continuation a lot, so it is indeed an important method for all sorts of NAG calculations. There is, however, more to NAG; for example, mixed methods combining numerical and symbolic approach, or numerical certification that a given numerical result is rigorously correct [1, 2].

We will also examine in greater detail how can be homotopic continuation, and NAG in general, utilised in certain optimisation and optimal control problems. For a calculation of one such problem, the program Bertini is used [4].

Bertini is one of the leading software for performing numerical algebraic geometry calculations [4, 5]. However, there are also alternatives, for example the library `HomotopyContinuation.jl` for Julia programming language [6]. Bertini's advantage (but also disadvantage, in a sense) is its specialisation to NAG, so it is effective, uses adaptive precision natively, and using it for solving particular polynomial systems or examining particular algebraic varieties is really simple [4].

2 Preliminaries and Notation

We briefly state here the notation being used, and also some relevant results from numerical linear algebra.

The space of all complex polynomials in variables $x_1, \dots, x_n$ is, as usual, denoted $\mathbb{C}[x_1, \dots, x_n]$. By $f \in \mathbb{C}_N[x_1, \dots, x_n]$, we mean that $f : \mathbb{C}^n \rightarrow \mathbb{C}^N$ and each f component f_j is polynomial, i.e. $f_j \in \mathbb{C}[x_1, \dots, x_n]$ for $j \in \{1, \dots, N\}$.

For $f \in \mathbb{C}_N[x_1, \dots, x_n]$, the symbol $\mathbb{V}(f) := f^{-1}(\{0\})$ means an affine algebraic variety, i.e. a Zariski-closed subset of the affine space $\mathbb{C}^n$ (the notation $\mathbb{A}^n$ for affine space is not used here). Sometimes we also work in projective space $\mathbb{CP}^n$ and homogeneous polynomials $F \in \mathbb{C}[x_0, x_1, \dots, x_n]$; then $\mathbb{V}(F)$ might denote a projective algebraic geometry in $\mathbb{CP}^n$, but this will be emphasised in the particular context.

The affine space $\mathbb{C}^n$ comes both with the usual Euclidean topology, but also with Zariski topology. When topological terms are used without a reference to the particular topology, we implicitly mean the standard Euclidean topology. This might be in contrast to algebraic geometry standard, but because NAG also utilises facts from the theory of complex analytic functions and numerical analysis, the Euclidean topology is often relevant.

Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^N$ be differentiable (or synonymously holomorphic, or complex analytic). Then we denote the Jacobi matrix of f in a point $x \in \mathbb{C}^n$ as $Df(x)$.

It is a standard linear algebra result that for a matrix $A \in \mathbb{C}^{m \times n}$ with rank $r \leq \min\{m, n\}$, there exist unique positive real values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ (the *singular values* of A) and some orthonormal vector sequences $(u_j)_{j=1}^r$ in $\mathbb{C}^n$, $(v_j)_{j=1}^r$ in $\mathbb{C}^m$, such that $A = \sum_{j=1}^r \sigma_j u_j v_j^*$, where v_j^* denotes the hermitian conjugate (so v_j^* is a linear form on $\mathbb{C}^n$) – this is the *singular value decomposition* (SVD) of A [7, 8].

The uniqueness of singular values justifies the definition of the *condition number* of A as

$$\kappa(A) := \sigma_1 / \sigma_r \geq 1.$$

For square matrices, this is the same as $\kappa(A) = \|A\| \cdot \|A^{-1}\|$, where $\|\cdot\|$ is the spectral norm

$$\|A\| := \max_{\|x\|=1} \|Ax\|, \quad \|v\| := \sqrt{\sum_{j=1}^n |v_j|^2},$$

which measures how much can the operator $v \mapsto Av$ stretch the unit sphere. This norm satisfies $\|Ax\| \leq \|A\| \cdot \|x\|$. Moreover, it holds that $\|A\| = \sigma_1$, the largest singular value of A [7, 8].

The condition number signifies in many contexts how numerically tame the matrix A is for various numerical linear algebraic computations. As an example (see [7], Proposition 5.3.1), assume the system $Ax = b$ with square regular A is being solved for unknown x , with $b \neq 0$. Now let the right hand side b be slightly perturbed as $b + \Delta b$, and denote $x + \Delta x$ the solution of the new system with A , $b + \Delta b$. Then from linearity, $A\Delta x = \Delta b$, and we have

$$\|b\| = \|Ax\| \leq \|A\| \cdot \|x\| \implies \frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|}, \quad \|\Delta x\| = \|A^{-1}\Delta b\| \leq \|A^{-1}\| \cdot \|\Delta b\|,$$

which combines into

$$\frac{\|\Delta x\|}{\|x\|} \leq \|A\| \cdot \|A^{-1}\| \frac{\|\Delta b\|}{\|b\|} = \kappa(A) \cdot \frac{\|\Delta b\|}{\|b\|}.$$

This estimate tells us that for small condition number $\kappa(A)$, the result of solving the system $Ax = b$ with perturbed right hand side does not differ much from the original solution; however, for large condition number, a slight perturbation of b can lead to a large change of the solution x (the problem is sensitive). This is numerically problematic due to slight perturbations being applied all the time due to the usage of rounding and finite precision arithmetics.

The following result (see [8], Theorem 5.8) gives a connection between almost-singular matrices and badly conditioned matrices:

Theorem 1 (Eckart, Young, Mirsky). *Let $A \in \mathbb{C}^{m \times n}$ be a rank $r \geq 1$ matrix and $A = \sum_{j=1}^r \sigma_j u_j v_j^*$ its SVD. Then the value*

$$\inf_{\substack{B \in \mathbb{C}^{m \times n} \\ \text{rank } B < r}} \|A - B\|$$

is equal to σ_r and attained for the matrix $B := \sum_{j=1}^{r-1} \sigma_j u_j v_j^$ of rank $r - 1$.*

For square nonsingular matrix $A \in \mathbb{C}^{n \times n}$ with SVD $A = \sum_{j=1}^n \sigma_j u_j v_j^*$, the previous theorem states that the distance A from the space of singular $n \times n$ matrices is

$$\sigma_n = \frac{\sigma_n}{\sigma_1} \cdot \sigma_1 = \frac{\|A\|}{\kappa(A)}.$$

Suppose $A : U \subseteq \mathbb{C}^m \rightarrow \mathbb{C}^{n \times n}$ is a continuous function, $s \in U$ and $A(s)$ is singular. Then from continuity, even if $A(x)$ are nonsingular on $U \setminus \{s\}$, they are near-singular, i.e. $\|A(x)\|/\kappa(A(x))$ is small. If also $\|A(x)\|$ is bounded on a neighbourhood of s (which is again due to continuity of A coefficients a fair assumption), then $\kappa(A(x))$ has to blow up into infinity as x approaches s . This will be relevant for dealing with singular points of affine varieties $\mathbb{V}(f)$, as in such points the Jacobian Df will be singular, so Df will be badly-conditioned on neighbourhood of such singularities.

3 Finding Isolated Roots Using Homotopic Continuation

In this section, we look at the problem of finding isolated roots of a given polynomial system. For an arbitrary function $f : \mathbb{C}^n \rightarrow \mathbb{C}^N$, we say that $x^* \in \mathbb{C}^n$ is an *isolated root* of f , if $f(x^*) = 0$ and f is nonzero on some punctured neighbourhood of x^* in $\mathbb{C}^n$.

When f is polynomial, such roots correspond to zero-dimensional components of the affine variety $\mathbb{V}(f)$.

Finding such roots is the cornerstone to describing the variety $\mathbb{V}(f)$ numerically, even if $\dim \mathbb{V}(f) > 0$. That is because the numerical approach restricts the way how we handle data – it is natural to track individual points $x \in \mathbb{C}^n$ numerically, whereas handling whole polynomials $g \in \mathbb{C}^{N'}[x_1, \dots, x_n]$, e.g. defining a positive-dimensional component of $\mathbb{V}(f)$, is more natural¹ for symbolic computational tools [1].

There are many numerical tools for solving nonlinear systems of the form $f(x) = 0$ with $f : \mathbb{C}^N \rightarrow \mathbb{C}^N$, e.g. Newton’s method or fixed point iteration method being commonly used [9, 10]. Let us take Newton’s method as an example of such a nonlinear equation solver to see both its advantages and disadvantages:

Definition 1. Let $f : \mathbb{C}^N \rightarrow \mathbb{C}^N$ be holomorphic. Pick an initial value $x_0 \in \mathbb{C}^N$. Then *Newton’s method* is given by the iterates

$$x_{k+1} := x_k - (Df(x_k))^{-1} \cdot f(x_k). \quad (1)$$

(For $f : \mathbb{R}^N \rightarrow \mathbb{R}^N$, the iterates are formally the same and f suffices to be differentiable.)

Let $x^* \in \mathbb{C}^n$ be an isolated root of f . If $Df(x^*)$ is nonsingular, then this sequence of Newton’s iterates converges to x^* for any choice of x_0 *close enough to x^** , and under mild additional conditions this convergence is quadratic, i.e. fast [10]. In other words, the basic convergence result is *local*. Moreover, even for polynomial f , the method can be sensitive to the choice of the initial value x_0 , as *Newton’s fractals* demonstrate [11].

Even if x_0 is chosen well and it converges to a root x^* of f , we would like to also find other roots of f . Choosing different x_0 might still happen to converge to x^* . If we have a set of already found roots $\{x^{*,1}, \dots, x^{*,\ell}\}$, we cannot really effectively determine if we found all the roots just by sampling more x_0 values not converging to new roots.

Thus, a different method for solving $f = 0$, $f \in \mathbb{C}_N[x_1, \dots, x_N]$ is applied in numerical algebraic geometry. It is the *homotopic continuation method*, having the strong advantage that it finds *all the roots* of f when performed correctly.

¹This is only partially true – if we find enough values from $\mathbb{V}(g)$, then we can interpolate these values and find the coefficients of g . This procedure can be indeed done numerically, with the numerical representation of g as a tuple of its coefficients. However, such an interpolation can computationally be very expensive, so for a particular computational problem dealing with $\mathbb{V}(g)$, it might be more suitable to use a different procedure without explicit computation of g . Moreover, note that even to find the samples of $\mathbb{V}(g)$, we need to work with individual points, so the importance of tracking individual points persists [2, 3].

3.1 General Structure of Homotopic Continuation

Assume now that $n = N$, so $f \in \mathbb{C}_N[x_1, \dots, x_N]$, for which we want to solve $f = 0$. The idea of homotopic continuation is to construct an easily-solvable system $g = 0$, $g \in \mathbb{C}_N[x_1, \dots, x_N]$, and then smoothly perturb this system to $f = 0$ while tracking how the solutions of $g = 0$ change during the perturbation [2, 3]. This "system perturbation" can be formalised by the following (more general than just polynomial) definition:

Definition 2. Let $f, g : \mathbb{C}^n \rightarrow \mathbb{C}^N$ be smooth functions. A function $h : \mathbb{C}^n \times [0, 1] \rightarrow \mathbb{C}^N$ is called a *smooth homotopy* of f and g , if h is smooth on $\mathbb{C}^n \times (0, 1)$ and $h(\cdot, 0) = f$, $h(\cdot, 1) = g$. We also use the notation $h_t := h(\cdot, t)$ for $t \in [0, 1]$.

In our case, we choose $g \in \mathbb{C}_N[x_1, \dots, x_N]$ which we can easily solve, and then we construct a homotopy h connecting g and f . Let x^0 be a solution of $g = 0$.

Suppose there is a smooth trajectory $\bar{x} : [0, 1] \rightarrow \mathbb{C}^N$ such that $\bar{x}(1) = x^0$ and for all $t \in [0, 1]$, $h(\bar{x}(t), t) = 0$. Then by differentiating w.r.t. t , we get an ordinary differential equation (ODE)

$$\frac{\partial h}{\partial t}(\bar{x}(t), t) + D_x h(\bar{x}(t), t) \cdot \frac{d\bar{x}}{dt}(t) = 0,$$

called the *Dauidenko differential equation* [3, 5].

This equation can be rewritten as

$$\frac{d\bar{x}}{dt} = -(D_x h(\bar{x}, t))^{-1} \frac{\partial h}{\partial t}(\bar{x}, t), \quad \bar{x}(1) = x^0, \quad (2)$$

which is an initial value problem, and thus known to have a unique solution under mild conditions for the right-hand side (in particular, continuity and local Lipschitz continuity w.r.t. x from the Picard-Lindelöf theorem) [12]. Finding this trajectory $\bar{x}$ is known as a *path tracking*.

The equation (2) can be discretised and numerically solved by vast number of numerical integration methods. Such methods are typically marching algorithms in the sense that they start in time $t = 1$ with known initial value $\bar{x}(1) = x^0$, and then iteratively decrease the time and calculate approximations of $\bar{x}(t_1), \bar{x}(t_2), \dots$, where $1 > t_1 > t_2 > \dots$ are positive [10].

Each such approximation $x^{[k]} \approx \bar{x}(t_k)$ is influenced by an error. However, we expect such an approximation to be close to the true value $\bar{x}(t_k)$, so we correct it by running Newton's method on the system $h(x, t_k) = 0$ with initial guess $x_0 := x^{[k]}$. This way, the approximation of $\bar{x}(t_k)$ is improved before another integration step is employed to get $x^{[k+1]} \approx \bar{x}(t_{k+1})$. This switching between marching in time and improving spatial approximation is known as *predictor* and *corrector steps*, respectively [3, 5].

A general homotopic continuation for the polynomial system $f = 0$, $f \in \mathbb{C}_N[x_1, \dots, x_n]$ is given by this recipe [5]:

1. Recast the problem into a square form, i.e. from now on, assume $f \in \mathbb{C}_N[x_1, \dots, x_N]$, giving a solution set in some sense related to the original solution set.
2. Choose an easily-solvable initial system g with $\dim \mathbb{V}(g) = 0$ and a smooth homotopy h s.t. $h(\cdot, 1) = g$ and $h(\cdot, 0) = f$. Find the (finite) solution set of $g = 0$.
3. Perform the numerical path tracking of the solution $\bar{x}$ of the ODE (2) up to some time $t = \varepsilon > 0$. This is achieved by switching between *predicting* a new approximation of $\bar{x}(t_{k+1})$ using numerical integration and previous approximation of $\bar{x}(t_k)$, and then *correcting* this new approximation via Newton's method applied to the system $h(x, t_{k+1}) = 0$.
4. When $t \leq \varepsilon$, apply an *endgame* to extract the value $\bar{x}(0)$ in a numerically robust way.

We have not discussed endgames so far. Their importance stems from the bare minimum condition on Newton's method – that is, that the matrix $Df(\bar{x}(0))$ should not be singular. If this Jacobi matrix is singular, it means that the path $\bar{x}$ converges to a *singular point* of $\mathbb{V}(f)$ [3, 5].

Numerically, singular $Df(\bar{x}(0))$ signals problems – both the predictor and corrector steps need to work with the Jacobi matrix, but as t approaches zero, $Df(\bar{x}(t))$ is closer and closer to a singular matrix, which leads to larger and larger errors due to rounding errors and sensitivity of near-singular matrices to small perturbations, as we discussed in the first section.

Thus, the path is usually not tracked to the final time $t = 0$, but it stops somewhere earlier and the value $\bar{x}(0)$ is obtained by different means, usually with the aid of complex analysis.

Luckily, $t = 0$ is most often the only problematic t value in which $\bar{x}$ can be singular. By "most often", it is meant that the typical homotopies used introduce some additional parameters, and for general values of these parameters, the resulting path $\bar{x}$ misses all the singularities. By general values, we mean the standard algebraic geometric term that such values form a nonzero Zariski-open set (i.e. a set dense in the ambient complex space) [3, 5]. This will be more discussed in the relevant parts of the following sections.

We will now explore each of the four components of homotopic continuation in more detail. Each of them can be studied somewhat independently, but together they give the complete picture of the continuation method.

We will for now assume that $f = 0$ is a square system, and discuss the case $n \neq N$ later on.

3.2 Homotopies

Let d_j denote the degree of f_j , where $j \in \{1, \dots, N\}$. By Bézout's theorem, $\mathbb{V}(f)$ consists of at most $D := d_1 \cdot d_2 \cdots d_N$ points in $\mathbb{C}^N$ (exactly D points when counting multiplicities and "solutions in infinity", i.e. new solutions after embedding $\mathbb{C}^N$ in the projective space $\mathbb{CP}^N$ and homogenising f) [3, 5, 13].

Without assuming any extra structure of f , this bound might be sharp, so our starting system $g = 0$ for the homotopic continuations should also have D roots. Moreover as we mentioned earlier, we would like to $g = 0$ be easily solvable. This motivates the choice $g_j(x) := x_j^{d_j} - 1$, which has the solution set

$$\mathbb{V}(g_j) = \{x \in \mathbb{C}^N \mid x_j = e^{2\pi i k/d_j}, k \in \{1, \dots, d_j\}\},$$

and thus

$$\mathbb{V}(g) = \bigcap_{j=1}^N \mathbb{V}(g_j) = \{(e^{2\pi i k_1/d_1}, e^{2\pi i k_2/d_2}, \dots, e^{2\pi i k_N/d_N}) \mid k_j \in \{1, \dots, d_j\}\},$$

i.e. j -th component is a d_j -th root of unity.

A natural homotopy connecting f and g is a line segment, i.e. a convex combination $(1-t)f + tg$. This is usually also enhanced by introducing complex parameter γ multiplying g , as it is given in the following definition [3, 5].

Definition 3. For $j \in \{1, \dots, N\}$, let $f_j \in \mathbb{C}[x_1, \dots, x_N]$ be of degree d_j , and denote $f := (f_1, \dots, f_N)$. By a *total degree homotopy* for f , we mean the homotopy

$$h_t := (1-t)f + t\gamma g,$$

where $g_j \in \mathbb{C}[x_1, \dots, x_N]$ is defined as $g_j(x) := x_j^{d_j} - 1$, and γ is a nonzero complex number.

The work [14] proves that for generic γ , this homotopy works:

Theorem 2 (Morgan, Sommese). *Let $f \in \mathbb{C}_N[x_1, \dots, x_N]$. For all but finite number of $\theta \in [0, 2\pi)$, the following holds: Let $\gamma := re^{i\theta}$ for any $r > 0$ and define the total degree homotopy as above. Then there exist smooth curves $\bar{x}_1, \dots, \bar{x}_k : (0, 1] \rightarrow \mathbb{C}^N$ such that for every $t \in (0, 1]$, the solution set $h(\cdot, t) = 0$ is given by the set $\{\bar{x}_1(t), \dots, \bar{x}_k(t)\}$. Moreover, for each root x^* of the system $f = 0$ with multiplicity $m \geq 1$, there exists exactly m paths $\bar{x}_{k_1}, \dots, \bar{x}_{k_m}$ converging to x^* as $t \searrow 0$.*

In actual computation, γ can be set as $\gamma := e^{i\theta}$ with random θ value, which will almost certainly miss the bad set of f for which the theorem does not hold. However, as systems emerging in applications are often real and expected to have interesting real

solutions, picking just $\gamma = 1$, i.e. a real path from 1 to 0, might not be generic enough; that is why we use a random complex factor γ .

The number of paths in total degree homotopy scales badly with increasing degrees and/or increasing size of the system. It is quite common that the studied system is not that much generic and has an extra structure. In those cases, different homotopies and starting systems can lead to much smaller number of paths while still ensuring that all solutions of $f = 0$ will be found. This has of course good performance implications, but the way how the initial system is constructed and solved might be more complicated and computationally expensive than for the total degree case [2, 3, 5].

3.3 Path Tracking

The path tracking corresponds to switching between prediction and correction steps as means to follow zeroes of a particular homotopy h .

A prediction step consists of a particular discretisation of the differential equation (2). A simplest numerical integrator discretising such an equation is the Explicit Euler method

$$x_{[0]}^{[k+1]} := x_{[0]}^{[k]} - \Delta t \cdot (D_x h(x_{[0]}^{[k]}, t))^{-1} \frac{\partial h}{\partial t}(x_{[0]}^{[k]}, t), \quad (3)$$

where $x_{[0]}^{[k]}$ is an already-computed approximation of $\bar{x}(t)$, Δt is a particular time step size and $x_{[0]}^{[k+1]}$ is a new approximation of $\bar{x}(t + \Delta t)$ [10]. This method is known to have first-order local discretisation error, i.e. if $x_{[0]}^{[k]}$ is precisely $\bar{x}(t)$, then

$$\frac{|\bar{x}(t + \Delta t) - x_{[0]}^{[k+1]}|}{|\Delta t|} = \mathcal{O}(|\Delta t|), \quad \Delta t \rightarrow 0.$$

In this sense, the method (3) is not very precise for numerical integration of ODEs, as during numerical ODE solving the individual step errors can accumulate into a large global error $|x_{[0]}^{[k]} - \bar{x}(t)|$, so it is usually preferable to use a higher-order numerical integrator. However, as a prediction step, (3) suffices to obtain a reasonable initial approximation of $\bar{x}(t + \Delta t)$, which can then be made more precise using the corrector. Additionally, it is computationally cheaper to calculate (3) than higher-order schemes' steps [3, 5, 10].

The correction step aims to improve the approximation $x_{[0]}^{[k+1]}$ of $\bar{x}(t + \Delta t)$, gotten from the prediction step. This approximation can be iteratively improved using Newton's method (1) on the equation $h(x, t + \Delta t) = 0$, which in this case yields

$$x_{[j+1]}^{[k+1]} := x_{[j]}^{[k+1]} - (D_x h(x_{[j]}^{[k+1]}, t + \Delta t))^{-1} \cdot h(x_{[j]}^{[k+1]}, t + \Delta t). \quad (4)$$

Using Newton's method is motivated by its fast convergence for $x_{[0]}^{[k+1]}$ near the true solution $\bar{x}(t + \Delta t)$ and its usage of only first-order derivative information – the differentials $D_x h$ need to be effectively computed anyway for timestepping the ODE (2).

Both number of Newton steps and the stepsize Δt are free parameters. In Bertini, the number of Newton steps is fixed and only Δt is adaptively controlled, so that convergence of each predictor-corrector step is ensured. Bertini also works with adaptive precision, calculating with more digits when the path passes through a badly-conditioned zone (e.g. near a singularity or a different path) [4, 5].

3.4 Intermezzo: Problematic Points in Path Tracking

The theorem 2 tells us that at least for the total homotopy, a path $\bar{x}_k$ starting in a root $\bar{x}_k(1) = x_k^0$ of g is well-behaved on $(0, 1]$. There are at least three things that might still go wrong:

1. Two paths $\bar{x}_k, \bar{x}_\ell$ might come very close to each other, i.e. $|\bar{x}_k(t_1) - \bar{x}_\ell(t_2)|$ is very small for some $t_1, t_2 \in (0, 1)$;
2. A path $\bar{x}$ might not converge to any finite x^* as $t \searrow 0$;
3. The root $x^* := \lim_{t \searrow 0} \bar{x}(t)$ of f for some path $\bar{x}$ might be a singular point of $\mathbb{V}(f)$.

The first problem is numerical in essence, as approximate path tracking of such paths might get confused at points $\bar{x}_k(t_1) \approx \bar{x}_\ell(t_2)$ which path it is actually following, thus a jump between paths might happen. Bertini mitigates this by employing adaptive floating point precision, which it can increase when it suspects that such a problematic branching point is nearby [4, 5].

The second problem is a symptom of not formulating the path tracking in the projective setting. Let F and G be homogenised versions of f and g by introducing a new variable x_0 , and let homotopy H connect F and G . Then straightforwardly all the computations can be done in the projective space $\mathbb{CP}^N$ with variables $[x_0 : x_1 : \dots : x_N]$ instead of the affine space $\mathbb{C}^N$ with variables $(x_1, x_2, \dots, x_N)$, just with the embedding $(x_1, \dots, x_N) \mapsto [1 : x_1 : \dots : x_N]$.

In the projective space, Bézout's theorem guarantees that if d_j denotes the degree of F_j (or equivalently f_j), then the number of solutions of $F = 0$ in $\mathbb{CP}^N$, also counting multiplicities, is equal to $D := d_1 \cdot d_2 \cdot \dots \cdot d_N$. In the context of total degree homotopy (which has D distinct roots, even after homogenisation) and theorem 2, there is a pairing of the roots of the original system $G = 0$ and the roots of $F = 0$, where to each multiplicity m root $x^* \in \mathbb{V}(F) \subseteq \mathbb{CP}^N$ corresponds exactly m roots of $G = 0$ whose paths converge to x^* . I.e. multiplicity m root $x^* \in \mathbb{V}(F)$ can be viewed as m copies of root x^* , which then means that both $\mathbb{V}(F)$ and $\mathbb{V}(G)$ have cardinality D , and moreover for each $x^* \in \mathbb{V}(F)$, there is a starting point $x^0 \in \mathbb{V}(G)$ and a path with boundary

points x^0, x^* ; therefore, the roots of F and G are in one-to-one correspondence and each path starting in $\mathbb{V}(G)$ has to converge to a root in $\mathbb{V}(F)$.

The diverging paths given by the second problem in the list thus have to converge to a point in infinity, i.e. some point $[0 : x_1^* : \dots : x_N^*]$ which does not correspond to any point at the affine variety $\mathbb{V}(f)$ (but is a solution of $F = 0$).

A standard way to deal with such paths diverging to infinity is to actually work in the projective space, and Bertini also does so. That way, the intermediate path values can be rescaled during the computation so that the values $x_1, \dots, x_N$ have reasonable magnitude, with the possibility that x_0 starts getting very close to zero. If that happens, it indicates that the path might be such a diverging one, i.e. converging to a projective root $[0 : x_1^* : \dots : x_N^*]$. This root might be of interest and can be path tracked, but it might also be useless for the particular application, in which case Bertini can discard such a path if x_0 values start getting too small [4, 5].

Finally, for the third problem of a path converging to a singular point of $\mathbb{V}(f)$, the endgames are meant to be used.

3.5 Endgames

Suppose we have a path $\bar{x} : (0, 1] \rightarrow \mathbb{C}^N$ solving the ODE (2) and satisfying $\lim_{t \searrow 0} \bar{x}(t) = x^* \in \mathbb{C}^N$, i.e. approaching a finite point as t goes to zero. From continuity, this x^* is a root of $h(x, 0) = 0$, which means x^* is a root of $f = 0$, and thus lies in $\mathbb{V}(f)$.

Let us investigate the Jacobi matrix $Df(x^*) \in \mathbb{C}^{N \times N}$. If this matrix is nonsingular, then the numerical path tracking of $\bar{x}$ can be performed all the way to $t = 0$ and x^* can be easily obtained.

However, if $Df(x^*)$ is singular (i.e. x^* is a singular point of $\mathbb{V}(f)$), Newton's method is not guaranteed to converge to x^* from neighbourhoods of x^* . Moreover, the numerical integration of (2) also gets numerically unstable, as its right-hand side depends on the inverse of the nearly-singular Df on a neighbourhood of x^* , which is numerically badly conditioned. Therefore, neither the predictor nor the corrector steps can be in general trusted to converge the numerical path to x^* .

Note that this is a numerical issue, as analytically $\lim_{t \searrow 0} \bar{x}(t) = x^*$. However, improving numerical precision only lets us get closer to $t = 0$, but for any finite precision there will be a neighbourhood of x^* where $Df(x^*)$ is so badly conditioned that such a precision does not suffice.

That is the reason why endgames are incorporated into homotopic continuation method (instead of ever-increasing precision to get really close to $t = 0$ and hoping that our approximations are close enough to x^*). Firstly, the path $\bar{x}$ is numerically tracked in the generally safe region $\{\bar{x}(t) \mid t \in [\varepsilon, 1]\}$, but after the computation gets close to

x^* , this value is extracted by different means [3, 5].

In [15], a version of the following theorem² is proven:

Theorem 3 (Morgan, Sommese, Wampler). *Let $T \subseteq \mathbb{C}$ be open such that $[0, 1] \subset T$. Let $h : \mathbb{C}^N \times T \rightarrow \mathbb{C}^N$ be holomorphic and $h(\cdot, 0) = f$, $h(\cdot, 1) = g$, with $f, g \in \mathbb{C}_N[x_1, \dots, x_N]$. Let $x^0, x^* \in \mathbb{C}$ satisfy $g(x^0) = 0$, $f(x^*) = 0$. Assume that there is a connected (complex) curve $K \subseteq \mathbb{C}^N \times T$ contained in $h^{-1}(\{0\})$ and that there is a smooth path $\bar{x} : [0, 1] \rightarrow \mathbb{C}^N$ with boundary points $\bar{x}(1) = x^0$, $\bar{x}(0) = x^*$, such that $\bar{x}([0, 1]) \times [0, 1] \subseteq K$ and for which $D_x h(\bar{x}(t), t)$ has full rank N for all $t \in (0, 1]$.*

Then, there exists $\varepsilon > 0$, positive integer c and a power series

$$Z(s) = \sum_{k=0}^{\infty} a_k s^k, \quad |s| \leq \varepsilon$$

centered in 0 with a radius of convergence at least ε , such that for any $s \in [0, \varepsilon]$, $Z(s) = \bar{x}(s^c)$.

Denoting $t := s^c$, $s := t^{1/c}$, we get that $\bar{x}$ can be on a neighbourhood $U_\varepsilon(0)$ extended into the convergent series

$$\bar{x}|_{\text{ext}}(t) = \sum_{k=0}^{\infty} a_k t^{k/c}, \quad |t| \leq \varepsilon^c. \quad (5)$$

Note that these series are formally *Puiseux series*, see e.g. page 133 in [13].

Theorem 3 can be utilised for multiple endgames, as described in the same work [15], but also in [3, 5].

The first such endgame is the *power series endgame*. This endgame samples $\bar{x}$ points near $t = 0$ and tries to interpolate these points by a polynomial given by truncation of (5) while assuming a particular c value. Typical c values tend to be reasonably small (they are positive integers, and they are also upper bounded by the multiplicity of x^* [3]), so they can be tested as $c = 1, 2, 3, \dots$. If any interpolation of a few sampled points with a particular c gives reasonably small errors, we assume we found the correct c and approximately correct truncation of (5) $\sum_{k=0}^J \hat{a}_k t^{k/c}$. We then get $x^* \approx \hat{a}_0$, as

$$x^* = \bar{x}(0) = \sum_{k=0}^{\infty} a_k \cdot 0^{k/c} = a_0.$$

A different endgame is the *Cauchy endgame*, which utilises the integral formula for holomorphic functions. This endgame assumes that $\bar{x}$ can be indeed extended from $[0, \varepsilon]$

²The original formulation deals more broadly with complex-analytic (i.e. holomorphic) homotopies. In this formulation, we specialise to homotopies connecting polynomials (which are holomorphic), and also adapt the notation to match with the rest of this work.

to the superset $U_\varepsilon(0) \subset \mathbb{C}$ such that $h(\bar{x}(t), t) = 0$ for any $t \in U_\varepsilon(0)$. This practically just means that we can smoothly move from $\bar{x}(\varepsilon)$ to $\bar{x}(t)$ with the above property, $t \in U_\varepsilon(0)$.

Then (5) holds on the whole neighbourhood $U_\varepsilon(0)$ and we can in particular sample values along the circle $\kappa : t(p) := \varepsilon e^{2\pi i p}$. Thus we get some samples along this curve, and we can also determine c – it is the smallest positive number of complete rotations along the circle κ for which the final sample returns to the original value. For example, if $c = 2$, then

$$\bar{x}(\varepsilon e^{2\pi i}) = \sum_{k=0}^{\infty} a_k \varepsilon^{k/2} e^{k\pi i} \neq \sum_{k=0}^{\infty} a_k \varepsilon^{k/2} = \bar{x}(\varepsilon),$$

but two revolutions along the circle will yield $\bar{x}(\varepsilon e^{4\pi i}) = \bar{x}(\varepsilon)$, so c is detected to be 2 (in this sense, $\bar{x}$ is not really a single valued function, just as a logarithm on $\mathbb{C} \setminus \{0\}$ is not a proper function and can change by smooth variation along a closed curve).

Finally, the value of the root x^* is recovered by the Cauchy integral formula applied for the circle κ and holomorphic function Z as

$$x^* = a_0 = Z(0) = \frac{1}{2\pi i} \oint_{\kappa} Z(s^c) ds,$$

where the right-hand side integral can be approximated using some numerical quadrature rule applied for the samples of $\bar{x}$ along the c revolutions of κ (corresponding to one revolution of Z along κ) [3, 5].

This careful endgame treatment of singularities can be computationally expensive. The Cauchy endgame, for example, needs us to follow new path just to extract the x^* value, so it can be more expensive than the previous path tracking along the original homotopy h for $t \in [\varepsilon, 1]$ itself. Also, adaptive precision of the floating point arithmetic needs to be in general employed, as it might happen that the so called *operational area of the endgame* $U_\varepsilon(0)$ where (5) can be used is too narrow that the numerical inaccuracies of following paths in this vicinity of 0 are too large in standard double precision.

It is however possible to estimate whether x^* is a singular point of $\mathbb{V}(f)$ by looking at the numerical conditioning of Df in $\bar{x}(t)$ as t approaches zero. This can be also indicated by seeing multiple paths clustering in small neighbourhood as $t \searrow 0$, in the essence of theorem 2, in which at least two paths must converge to x^* if it is singular (and thus of multiplicity at least 2).

If x^* seems to be a nonsingular point, then the path tracking with Newton's iterations can be straightforwardly used to find $x^* = \bar{x}(0)$. Some applications might not be interested in the singular isolated solutions of $f = 0$, so they can stop the path tracking without performing endgames if they suspect the path is approaching a singularity [3].

3.6 Parametric Continuation

Suppose now that we are trying to solve $f(x, \theta) = 0$, where $f \in \mathbb{C}_N[x_1, \dots, x_N, \theta_1, \dots, \theta_p]$ and we interpret θ as parameter values, on which f depends *polynomially*. This gives us a family of square polynomials $\mathcal{F} := \{f_\theta \mid \theta \in \mathbb{C}^p\}$, where $f_\theta := f(\cdot, \theta)$.

Central will be the following theorem, formulated as theorem 7.1.1 in [3]:

Theorem 4. *Let $f \in \mathbb{C}_N[x_1, \dots, x_N, \theta_1, \dots, \theta_p]$. For $\theta \in \mathbb{C}^p$, denote $r(\theta)$ the number of nonsingular solutions of $f_\theta = 0$, which we assume to be finite for some θ value. Then there exists a nonempty Zariski-open subset G of $\mathbb{C}^p$ such that the following holds:*

1. *The function r is constant on G ;*
2. *For $\theta \notin G$, $r(\theta)$ is strictly smaller than the constant value of r on G ;*

Suppose moreover that $\theta_0, \theta_1 \in G$ and $\varphi : [0, 1] \rightarrow G \subseteq \mathbb{C}^p$ is a smooth curve s.t. $\varphi(0) = \theta_0$, $\varphi(1) = \theta_1$. Form the homotopy $h(x, t) := f(x, \varphi(t))$. Then the paths starting in nonsingular points of $\mathbb{V}(f_{\theta_1})$ and following the homotopy h are smooth, nonsingular and as t approaches zero, these paths converge to all the nonsingular points of $\mathbb{V}(f_{\theta_0})$.

Theorem 4 guarantees us that a generic member of the family $\mathcal{F}$ defines qualitatively the same variety, and if we solve $f_{\theta_0} = 0$ for one particular generic $\theta_0 \in \mathbb{C}^p$, then we can via continuation in the parameter space solve any $f_\theta = 0$ for generic θ (i.e. not lying in the proper affine subvariety $\mathbb{C}^p \setminus G$ of $\mathbb{C}^p$).

This is mainly useful because of the point 1 (and also 3) in theorem 4 – if we choose f_{θ_0} as the initial system for homotopic continuation, then we follow just the necessary paths, so the procedure is as effective as it can. This is in contrast with following a total degree homotopy, which follows a large number of paths given by the theoretical maximal cardinality of $\mathbb{V}(f_{\theta_0})$ via Bézout’s theorem, even though this polynomial system might have much lesser number of solutions (in which case the remaining total homotopy paths diverge to infinity and just waste computational time) [3, 5].

Therefore, if the particular numerical application needs solving many systems $f_\theta = 0$ for various parameter values $\theta \in \mathbb{C}^p$, then it suffices to:

1. Perform one *offline* run, e.g. with total degree homotopy, to find the solution set of $f_{\theta_0} = 0$ for a chosen generic $\theta_0 \in \mathbb{C}^p$ (in practise, randomly generated);
2. For subsequent runs of solving $f_\theta = 0$, form a parameter space homotopy connecting θ_0 and θ and follow the paths to find regular values of $\mathbb{V}(f_\theta)$. For generic θ , this will give all the roots. According to [3] (lemma 7.1.2), it suffices to use $\varphi_\theta(t) := t\theta_0 + (1 - t)\theta$ and track paths in the homotopy $h_\theta(x, t) := f(x, \varphi_\theta(t))$. This gives an *online* run, which is expected to be computationally cheaper than the initial offline run.

3.7 Recasting Nonsquare Systems to a Square Form

Let $f \in \mathbb{C}_N[x_1, \dots, x_n]$. The common homotopic continuation numerical methods require $n = N$ in order to numerically solve $f = 0$. If $n = N$, we call $f = 0$ a *square system*, whereas a system with $n > N$ is said to be *underdetermined* and system with $n < N$ *overdetermined*.

One reason for working with square systems is motivated by the idea that such systems are expected to have only isolated solutions, whereas underdetermined systems are more expected to have positive-dimensional components and overdetermined systems possibly no solutions at all (that is e.g. the typical behaviour for the linear systems $f(x) = b$). A more practical reason for enforcing $f = 0$ being squared is that the typical homotopic continuation methods used assume invertible Jacobi matrix Df , which makes sense mainly in the case when this matrix is square.

For overdetermined systems, *randomisation* is being used, in which the system is recasted into a square system consisting of n linear combinations of the original polynomials [3, 5]. The motivation for this is the following Bertini-type theorem ([3], theorem 13.5.1), which states that such a new square system is generally tightly connected to the original overdetermined system.

Theorem 5. *Let $f \in \mathbb{C}_N[x_1, \dots, x_n]$. Let $X \subseteq \mathbb{C}^n$ be an irreducible Zariski-closed set and $k < N$ a positive integer. Then there exists a nonempty Zariski-open set $U \subseteq \mathbb{C}^{k \times N}$ such that for $\Lambda \in U$, the following holds:*

1. *If $\dim X > n - k$, then X is an irreducible component of $\mathbb{V}(f)$ if and only if it is an irreducible component of $\mathbb{V}(\Lambda f)$;*
2. *If $\dim X = n - k$, then X being an irreducible component of $\mathbb{V}(f)$ implies that it is also an irreducible component of $\mathbb{V}(\Lambda f)$;*
3. *If X is an irreducible component of $\mathbb{V}(f)$, then its multiplicity as a solution of $f = 0$ is lesser than or equal to its multiplicity as a solution of $\Lambda f = 0$. For multiplicity 1 as a solution of $f = 0$, the multiplicity of X as a solution of $\Lambda f = 0$ is also 1.*

In particular for an overdetermined system f and $k := n < N$, we get that for a generic $\Lambda \in \mathbb{C}^{n \times N}$, Λf is a square system of size N . Then, the positive-dimensional components solutions of $f = 0$ and $\Lambda f = 0$ match, and $\mathbb{V}(\Lambda f)$ contains all the isolated solutions ($\dim X = 0$) of $f = 0$, so solving the square system $\Lambda f = 0$ in order to recover solutions of $f = 0$ makes sense.

More interesting situation is when the system is underdetermined, i.e. $n > N$. Typically, such system defines a positive-dimensional affine variety. Numerically, these

are represented by taking generic $L \in \mathbb{C}_{n-N}[x_1, \dots, x_n]$ linear and finding the solution set of $(f, L) = 0$, which is a square system. Let this solution set be finite and denote it W . Then (f, L, W) is called a *witness set* of f – it gives particular solutions of f when sliced by the linear variety $\mathbb{V}(L)$ [3, 5].

The advantage of witness sets is that even though they represent just a particular finite set of points on a connected component A of $\mathbb{V}(f)$, they can be effectively used to track different points on A using homotopies $(f, (1 - t)L + tH)$, where $H \in \mathbb{C}_{n-N}[x_1, \dots, x_n]$ is a linear polynomial.

Bertini handles this underdetermined case of $f = 0$ quite differently than square systems, with different structure of outputs. It lets a user to find an irreducible decomposition of $\mathbb{V}(f)$, with witness sets *witnessing* the particular irreducible component. Using homotopic continuation on those particular witness sets, the smooth paths stay in their particular irreducible components, which enables *membership testing*: If we want to find whether $p \in A$, where A is a component of $\mathbb{V}(f)$, we can construct a linear polynomial H containing p , and then track paths of the witness set of A by smoothly deforming the original linear witness polynomial L into H as $(f, (1 - t)L + tH) = 0$. If the final solution set in $t = 0$ contains p , then $p \in A$ (otherwise, p should not be path-connected to the witness set of A) [3, 4, 5].

3.8 Homotopic Continuation in Bertini

Using Bertini for homotopic continuation is very simple and is explained in [4] and [5]. Each program is given by an **input** file, which firstly contains optional configurations, and then the definition of constants, parameters, variables and target polynomials. Such a structure can look like this:

```
CONFIG
END;
INPUT
function f, g;
variable_group x, y;
f = x^2 - y*x;
g = x^3*y + y^2 - 2;
END;
```

In this example, no configuration is used, and the program specifies that we want to find the affine variety $\mathbb{V}(x^2 - yx, x^3y + y^2 - 2)$. Such a program can be simply run as **bertini input** from the command line. This generates many informative files about the continuation and the resulting solutions.

By default, Bertini will embed $\mathbb{C}^2$ into $\mathbb{CP}^2$ by introducing a third variable and homogenising f and g , and then employ total degree homotopy for this problem. It will also stop following any paths that seem to diverge into infinity. This can be overridden by a particular configuration, but even then Bertini separates finite solutions and solutions in infinity.

If we instead made the homogenisation ourselves by setting $f = x^2 - yx$, $g = x^3y - y^2z^2 - 2z^4$ and specifying `hom_variable_group x, y, z` instead of `variable_group x, y`, Bertini would work in the space $\mathbb{CP}^2$ from the start, with no discarding of "infinite solutions" or separating finite and infinite solutions, as the problem is a priori projective and not affine.

A simple parametric program can look like this:

```
CONFIG
PARAMETERHOMOTOPY: 1;
END;
INPUT
function f, g;
variable_group x, y;
parameter: r;
f = x^2 - r*y*x;
g = x^3*y + r*y^2 - 2;
END;
```

The setting `PARAMETERHOMOTOPY: 1;` specifies that this is the offline phase, so a random parameter value r should be sampled. Bertini will do so, saving it into `start_parameters` file, and then Bertini runs a total degree homotopy on the given system.

If one wants to perform the online stage of parametric continuation, the resulting `nonsingular_solutions` file should be moved to `start`, which is a file that specifies the solutions of the starting system of the homotopy. After that, the desired parameter values have to be listed in a file `final_parameters` (in the order in which they were declared in `input` file), and in the program, one needs to change the configuration `PARAMETERHOMOTOPY` from 1 to 2. This way, Bertini will run continuation in the parameter space from the `start_parameters` values to `final_parameters` values, starting paths from the solutions found in offline run and saved in `start` file.

4 Applications

There are many fields where the methods of numerical algebraic geometry can be utilised. It is, however, plausible that some of these fields have not stumbled upon NAG yet, as algebraic geometry is not a part of mathematics used extensively by applied mathematicians (who often focus more on analysis, linear algebra and statistics).

Among the standard and often cited fields utilising NAG is kinematics, i.e. design and analysis of mechanisms. For example, a robotic arm might be composed of several joints, in which specific angle values can be controlled in order to move the tip of the arm into a particular location. Many questions about such systems can be answered with the knowledge how a certain affine algebraic variety looks like – for the example of a robotic arm, even though the relevant values seem to be some angles, their cosines and sines suffice to describe the arm’s geometry, and then $(\cos \omega, \sin \omega)$ values can be replaced by (c, s) in the governing equations, with additional polynomial condition $c^2 + s^2 = 1$ [1, 16].

NAG was also in some cases applied to numerical solving of nonlinear ordinary and partial differential equations. Discretisations of such differential equations sometimes lead to polynomial systems, for which homotopic continuation can be applied. These systems’ number of variables increases with the fineness of underlying discretisation mesh, so in order to get accurate solution, a large polynomial system has to be solved. In some cases, application of NAG to nonlinear differential equations gave new candidates for solutions, which were previously unknown [5].

In this sense, experimentation with NAG can help for a rigorous mathematical research (in searching for patterns and formulation of conjectures, for example). Of course, in this essence is NAG relevant for researchers of algebraic geometry, as they can utilise NAG to explore particular varieties effectively.

As a final example of NAG application is optimal control. This is the application we investigate more in the following text.

4.1 Polynomial Optimisation and Optimal Control

4.1.1 Optimisation and Polynomial Optimisation

A general optimisation problem is formulated [17, 18] as

$$\text{minimise } J(x) \text{ subject to } g(x) \leq 0, \ h(x) = 0. \quad (6)$$

Here, $J : \mathbb{R}^n \rightarrow \mathbb{R}$ is the *cost function*, $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ are the *inequality constraints* and $h : \mathbb{R}^n \rightarrow \mathbb{R}^\ell$ are the *equality constraints*. The inequality $g(x) \leq 0$ means componentwise

inequality, i.e. $g_j(x) \leq 0, \forall j \in \{1, \dots, m\}$. The constraints define the *feasible set*

$$\Omega := \{x \in \mathbb{R}^n \mid g(x) \leq 0, h(x) = 0\}.$$

If J, g and h are differentiable and $x^* \in \Omega$ is a (local) minimum of J w.r.t. Ω , then under additional conditions³ on the set Ω , there exist $\mu^* \in \mathbb{R}^m$ and $\lambda^* \in \mathbb{R}^\ell$ such that the triplet (x^*, μ^*, λ^*) satisfies the *Karush–Kuhn–Tucker (KKT) conditions* (see [17], Theorem 12.1):

$$\begin{aligned} \nabla J(x^*) + \sum_{j=1}^m \mu_j^* \nabla g_j(x^*) + \sum_{j=1}^\ell \lambda_j^* \nabla h_j(x^*) &= 0, \\ h(x^*) &= 0, \\ \mu_j^* g_j(x^*) &= 0, \quad j \in \{1, \dots, m\}, \\ g(x^*) &\leq 0, \\ \mu^* &\geq 0. \end{aligned} \tag{7}$$

Now assume that J, g and h are polynomial. Then the first three equations in (7) form a polynomial system in $N := n + m + \ell$ unknowns x^*, μ^*, λ^* , with $n + \ell + m$ polynomial equations. Therefore, these KKT conditions form a square polynomial system. If this system has isolated solutions only, then those finitely many solutions can be found using homotopic continuation. Each of them can be then checked whether it satisfies the inequalities of (7). Finally, the found points satisfying all KKT conditions can be compared in terms of evaluation of J in them – the smallest one(s) can be then deduced to be optimal x^* (assuming that the original problem has a global minimum, which is true for example for bounded, i.e. compact Ω , as J is continuous).

The only really strong assumption we made is the one that J, g and h are polynomial. Otherwise, in optimisation numerical methods for general problem (6), both existence of a minimiser x^* and satisfied constraint qualifications in x^* are typically assumed in order to exploit the deduced necessary KKT conditions and find the minimiser.

Moreover, these methods are often local in nature and require usage of clever heuristics and regularisations in order to obtain global convergence. However by restricting to polynomial optimisation, we can ensure global convergence simply by using homotopic continuation.

³Those are the *constraint qualifications* (CQ). One of the most common CQ is the *linear independence constraint qualification* (LICQ) which requires that all vectors $\nabla h_j(x^*)$ and active vectors $\nabla g_j(x^*)$ are together linearly independent. By active vectors, we mean $\nabla g_j(x^*)$ such that $g_j(x^*) = 0$ (i.e. we omit those inequality constraints which are strict in x^*).

4.1.2 Optimal Control

For motivation of NAG to optimal control, see [19] and [20]. In optimal control, the goal is to, broadly, set some parameter values in order to force a given parametric dynamical system to behave in optimal way. Such dynamical system can be either discrete or continuous, so it is hard to give a unified mathematical description of the problem.

An example can be a car moving from $x_0 = 0$ to $x_1 = L$ with a parameter being the acceleration applied in a given time $a(t)$. The underlying continuous dynamical system is

$$\begin{aligned}\frac{dx}{dt}(t) &= v(t), & x(0) &= x_0 \\ \frac{dv}{dt}(t) &= a(t), & v(0) &= 0,\end{aligned}$$

where $x(t)$ and $v(t)$ are the car's position and velocity in time t , respectively. The optimal control goal can be, for example, to find optimal function a s.t. the time T for which $x(\tau) = L$ is minimal. This gives an optimal control problem. If we discretised these differential equations on a finite mesh, we would get a discrete dynamical system with recurrences between x_k, v_k, a_k, k being a particular time step. The optimisation problem can be also constrained, for example by requiring $v(t) \leq v_{\max} = \text{const}$, i.e. velocity not exceeding a given maximal speed.

Interesting is the problem of *model predictive control* (MPC), in which the optimisation problems are being solved during the evolution of the dynamical system (i.e. *online*) and parameters adjusted in real-time during the process. In the very simple car scenario, this could be that in a particular time τ , the next values of a on $[\tau, \tau + \Delta t]$ would be chosen by solving an optimisation subproblem, which would use some additional information about the system's behaviour in times $[0, \tau]$. An example could be the constraint $v_{\max}$ not being constant, but unpredictably changing with time, for example due to traffic jams. Thus the optimisation subproblems in various τ would change as the maximal speed would be measured in τ [18].

A more realistic setting for MPC can be a guidance system of a rocket/satellite or a power plant reacting to the load in the electric network. In order to MPC being applicable, the online optimisation problem has to be solved fast – otherwise, the found solution does not have to be relevant anymore, because significant time passed after the start of calculations and dynamical system evolved in the meantime.

This naturally motivates the use of parametric continuation, if the MPC optimisation subproblems are polynomial. Firstly, MPCs (or more broadly, optimal problems)

are by definition parametrised. If this parametrisation is also polynomial, then a first, more costly offline run can be performed before the MPC has to be employed in the particular application. Then, the individual real-time MPC optimisation subproblems can be cast into the algebraic formulation using the KKT conditions (7), and these systems can be solved with the much more effective online runs using parametric continuation and the precomputed offline result.

4.1.3 Experiment

To demonstrate optimal control in the setting of NAG, we consider a modified Lotka-Volterra model inspired by [21]:

$$\begin{aligned}\frac{dx}{dt} &= x \cdot (\alpha - u) - \beta xy, \\ \frac{dy}{dt} &= \delta xy - \gamma y.\end{aligned}\tag{8}$$

With $u = 0$, this corresponds to a standard Lotka-Volterra "predator-prey" model, where for $\alpha, \beta, \gamma, \delta$ nonnegative constants, the unknown functions x and y correspond to population sizes of predators vs. preys, respectively. The interaction terms $-\beta xy$ and δxy correspond to a decrease of prey numbers, or respectively to increase of predators, due to predators hunting prey.

The function u is the control parameter, which is in [21] located in the second equation and used to model a human hunting of the predators. By inclusion of this term into the first equation, we would like to model a hunting of prey in order to maximise the functional

$$\tilde{J}(x, u) := \int_0^T x(t)u(t)dt - \frac{\kappa}{2} \int_0^T u^2(t)dt$$

for some $T > 0$, without reducing the population too much, i.e. $x(t) \geq \varepsilon$ for some threshold $\varepsilon > 0$ and $t \in [0, T]$. Here, $\kappa > 0$, the first term of the functional corresponds to the yield by human-hunting of prey, and the second term gives regularisation (so that the behaviour is more interesting than just taking u very large in the beginning and then zero – moreover, such behaviour would be badly approximated after discretising).

In order to formulate this problem in discrete setting, we use the Explicit Euler method to write

$$\begin{aligned}x_{j+1} &= x_j + \Delta t \cdot (\alpha - u_j)x_j - \Delta t \cdot \beta x_j y_j, \\ y_{j+1} &= y_j + \Delta t \cdot \delta x_j y_j - \Delta t \cdot \gamma y_j.\end{aligned}$$

New function we wish to maximise is

$$J(x, u) := \sum_{j=0}^{M-1} x_j u_j - \frac{\kappa}{2} \sum_{j=0}^{M-1} u_j^2.$$

Equivalently, we wish to minimise $-J$.

The MPC setting here is this: Suppose we are in a step (x_k, y_k) after applying controls $(u_0, u_1, \dots, u_{k-1})$ and following trajectory $((x_0, y_0), (x_1, y_1), \dots, (x_{k-1}, y_{k-1}))$. We choose a positive integer N and try to solve the optimal control problem only for next few values $((x_{k+1}, y_{k+1}), \dots, (x_{k+N}, y_{k+N}))$. This will give, after applying the KKT conditions (7), much smaller polynomial system than if we tried to compute the optimal control over a whole time period $[0, T]$ (with fine-enough mesh Δt). After we find optimal solution, we take only its first values u_k, x_{k+1}, y_{k+1} as the next step, move to it and again solve new optimisation subproblem.

In phase k , we know (x_k, y_k) and we need to minimise

$$-\sum_{j=0}^{N-1} x_{k+j} u_{k+j} + \frac{\kappa}{2} \sum_{j=0}^{N-1} u_{k+j}^2$$

over variables $x_{k+1}, \dots, x_{k+N}, y_{k+1}, \dots, y_{k+N}, u_k, \dots, u_{k+N-1}$ subject to

$$\begin{aligned} x_{k+j} &\geq \varepsilon, \quad j \in \{1, \dots, N\}, \\ x_{k+j+1} &= x_{k+j} + \Delta t \cdot (\alpha - u_{k+j}) x_{k+j} - \Delta t \cdot \beta x_{k+j} y_{k+j}, \quad j \in \{0, \dots, N-1\}, \\ y_{k+j+1} &= y_{k+j} + \Delta t \cdot x_{k+j} y_{k+j} - \Delta t \cdot \gamma y_{k+j}, \quad j \in \{0, \dots, N-1\}. \end{aligned}$$

In the formalism of (6), we have

$$\begin{aligned} g_{k+j}(u, x, y) &:= \varepsilon - x_{k+j}, \quad j \in \{1, \dots, N\}, \\ h_{k+j}^x(u, x, y) &:= x_{k+j+1} - x_{k+j} - \Delta t \cdot (\alpha - u_{k+j}) x_{k+j} + \Delta t \cdot \beta x_{k+j} y_{k+j}, \quad j \in \{0, \dots, N-1\}, \\ h_{k+j}^y(u, x, y) &:= y_{k+j+1} - y_{k+j} - \Delta t \cdot x_{k+j} y_{k+j} + \Delta t \cdot \gamma y_{k+j}, \quad j \in \{0, \dots, N-1\}. \end{aligned}$$

After calculating

$$-\nabla J + \sum_{j=0}^{N-1} \lambda_{k+j} \nabla h_{k+j}^x + \sum_{j=0}^{N-1} \lambda'_{k+j} \nabla h_{k+j}^y + \sum_{j=1}^N \mu_{k+j} \nabla g_{k+j},$$

we get the following equality KKT conditions (7):

$$\begin{aligned}
\mu_{k+j}(\varepsilon - x_{k+j}) &= 0, & j \in \{1, \dots, N\}, \\
x_{k+j+1} - x_{k+j} - \Delta t \cdot (\alpha - u_{k+j})x_{k+j} + \Delta t \cdot \beta x_{k+j}y_{k+j} &= 0, & j \in \{0, \dots, N-1\}, \\
y_{k+j+1} - y_{k+j} - \Delta t \cdot x_{k+j}y_{k+j} + \Delta t \cdot \gamma y_{k+j} &= 0, & j \in \{0, \dots, N-1\}, \\
\kappa u_j - x_j + \Delta t \cdot \lambda_j x_j &= 0, & j \in \{0, \dots, N-1\}, \\
-u_j + \lambda_{j-1} + \lambda_j(-1 - \Delta t \cdot (\alpha - u_j) + \Delta t \cdot \beta y_j) - \Delta t \cdot \delta y_j \lambda'_j - \mu_j &= 0, & j \in \{1, \dots, N-1\}, \\
\lambda_{N-1} - \mu_N &= 0, \\
\Delta t \cdot \lambda_j \beta x_j + \lambda'_{j-1} + \lambda'_j(-1 - \Delta t \cdot \delta x_j + \Delta t \cdot \gamma) &= 0, & j \in \{1, \dots, N-1\}, \\
\lambda'_{N-1} &= 0.
\end{aligned} \tag{9}$$

The first three equations correspond to the constraints, the fourth is given by the u -parts of gradients, the fifth and sixth by the x -parts of gradients and the last two by the y -parts of gradients.

This is a parametrised polynomial system of $6N$ equations in $6N$ unknowns. For calculation in Bertini, we choose $N = 3$, yielding a square polynomial system of size 18. We handle the model parameters as parameters, i.e. we do not set any constant values to $\alpha, \beta, \gamma, \delta, \Delta t, \kappa, x_k, y_k$. This is possible because the system (9) is polynomial in these parameters. It also leads to a more powerful MPC system, which can adapt to changes of various parameters in time (i.e. if some of the $\alpha, \beta, \gamma, \delta$ parameters were time-dependent, or if we wanted to dynamically change the regularisation parameter κ).

The first, offline phase of parametric homotopic continuation follows 8192 paths given by the corresponding total degree homotopy for system (9). This process takes about half an hour on our laptop, but could be made faster using parallelisation on many-core computational cluster (each path being followed by different computational core).

The point is that this offline phase is computationally expensive even for small $N = 3$, but if this MPC was used in practise, this would not be of much concern, as long as the online phase is fast (so even slightly larger N values would be fine).

In the offline phase, Bertini found 34 solutions, none of which is singular. That means that the rest of $8192 - 34$ paths diverged to infinity. Now, each online phase is performed by setting desired parameter values and running parametric homotopic continuation from the offline system with the found 34 nonsingular solutions. This is orders of magnitude faster process, because only 34 paths need to be tracked.

Suppose we set $\Delta t = 0.1$, $\alpha := 0.4$, $\beta := 0.2$, $\gamma := 0.6$, $\delta := 0.3$, $\kappa := 0.1$, $\varepsilon := 0.2$,

$x_0 := 0.7$ and $y_0 := 0.4$. Running first online phase, we find solutions

$$(x_1, x_2, x_3, y_1, y_2, y_3, \lambda_0, \lambda_1, \lambda_2, \lambda'_0, \lambda'_1, \lambda'_2, \mu_1, \mu_2, \mu_3).$$

Each such solution needs to be checked whether it also satisfies the inequality KKT conditions (7), i.e. $x_j \geq \varepsilon$ and $\mu_j \geq 0$. If there are multiple such valid points, we should choose the solution giving optimal value of the functional J . After finding such a point, we extract (x_1, y_1) , set $k := 1$ and new online phase can be applied. This is practically done by changing the original parameter values (x_0, y_0) to newly found (x_1, y_1) . If some other parameter depended on time, we would also change its value before running new online continuation.

For our parameters, the found solution is given in the following table:

k	0	1	2	3	4	5	
x_k	0.7	0.520	0.404	0.327	0.277	0.237	0.207
y_k	0.4	0.384	0.367	0.349	0.332	0.315	0.298
u_k	2.896	2.550	2.235	1.995	1.777	1.601	—

In figure 1, 100000 random control vectors $u = (u_0, u_1, \dots, u_5)$ were (in Python) sampled uniformly randomly from $[0, 2]$ such that the dynamical system obeyed the constraints $x_j \geq \varepsilon$. The figure shows a histogram of the functional J values along these randomly controlled trajectories. The orange bar shows the optimal value which we found using (9) and Bertini.

We see that we really found the optimal control for the given problem. We thus solved an optimisation problem by a *globally convergent method*, i.e. there was no need for careful initial guesses how the optimal control should look like, as it is so in local optimisation methods. Moreover, by one offline run, a whole *family* of problems of the form (8) were essentially solved, as change of parameters just means that a quick online continuation has to be performed.

5 Conclusion

In this work, we gave a introductory exposition to what numerical algebraic geometry is. We described the essence of homotopic continuation method, its advantages and practical usage in Bertini software. We also showed how NAG can be used to solve problems of optimal control.

We believe the methods of NAG have a very sound performance and robustness and hope they will become a more standard tool in the applied and computational mathematics communities.

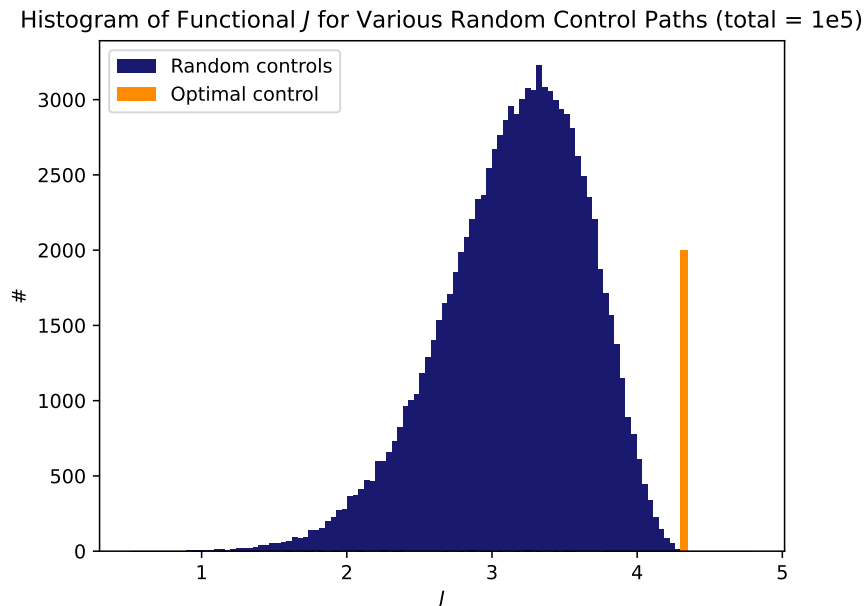


Figure 1: Values of J for randomly controlled paths vs the control found using Bertini.

References

- [1] Jonathan D. Hauenstein and Andrew J. Sommese. “What is numerical algebraic geometry?” In: *Journal of Symbolic Computation* 79 (2017). SI: Numerical Algebraic Geometry, pp. 499–507. ISSN: 0747-7171. DOI: <https://doi.org/10.1016/j.jsc.2016.07.015>.
- [2] Alicia Dickenstein. *Solving polynomial equations*. Springer, 2005.
- [3] Andrew John. Sommese and Charles Weldon Wampler. *The numerical solution of systems of polynomials arising in engineering and science*. eng. 1st ed. Hackensack, NJ: World Scientific, 2005. ISBN: 9786611880828.
- [4] Daniel J. Bates et al. *Bertini: Software for Numerical Algebraic Geometry*. Available at bertini.nd.edu with permanent doi: [dx.doi.org/10.7274/R0H41PB5](https://doi.org/10.7274/R0H41PB5).
- [5] Daniel J Bates et al. *Numerically solving polynomial systems with Bertini*. SIAM, 2013.
- [6] Paul Breiding and Sascha Timme. “HomotopyContinuation.jl: A Package for Homotopy Continuation in Julia”. In: *International Congress on Mathematical Software*. Springer. 2018, pp. 458–465.
- [7] Grégoire Allaire and Sidi Mahmoud Kaber. *Numerical linear algebra*. Springer, 2008.
- [8] Lloyd N Trefethen and David Bau. *Numerical linear algebra*. SIAM, 2022.

- [9] Francis Begnaud Hildebrand. *Introduction to numerical analysis*. Courier Corporation, 1987.
- [10] Endre Süli and David F Mayers. *An introduction to numerical analysis*. Cambridge university press, 2003.
- [11] ML Sahari and I Djellit. “Fractal newton basins”. In: *Discrete Dynamics in Nature and Society* 2006.1 (2006), p. 028756.
- [12] Wolfgang Walter. *Ordinary differential equations*. Vol. 182. Springer Science & Business Media, 2013.
- [13] Igor Rostislavovich Shafarevich and Miles Reid. *Basic algebraic geometry*. Vol. 2. Springer, 1994.
- [14] Alexander Morgan and Andrew Sommese. “A homotopy for solving general polynomial systems that respects m-homogeneous structures”. In: *Applied Mathematics and Computation* 24.2 (1987), pp. 101–113. ISSN: 0096-3003. DOI: [https://doi.org/10.1016/0096-3003\(87\)90063-4](https://doi.org/10.1016/0096-3003(87)90063-4). URL: <https://www.sciencedirect.com/science/article/pii/0096300387900634>.
- [15] Alexander P Morgan, Andrew J Sommese, and Charles W Wampler. “A power series method for computing singular solutions to nonlinear analytic systems”. In: *Numerische Mathematik* 63.1 (1992), pp. 391–409. DOI: <https://doi.org/10.1007/BF01385867>.
- [16] Charles W. Wampler and Andrew J. Sommese. “Numerical algebraic geometry and algebraic kinematics”. In: *Acta Numerica* 20 (2011), pp. 469–567. DOI: 10.1017/S0962492911000067.
- [17] Jorge Nocedal. *Numerical optimization*. 2006.
- [18] Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.
- [19] Daniel J Bates, Ioannis A Fotiou, and Philipp Rostalski. “A numerical algebraic geometry approach to nonlinear constrained optimal control”. In: *2007 46th IEEE Conference on Decision and Control*. IEEE. 2007, pp. 6256–6261.
- [20] Philipp Rostalski et al. “Numerical algebraic geometry for optimal control applications”. In: *SIAM Journal on Optimization* 21.2 (2011), pp. 417–437.
- [21] Narcisa Apreutesei and Gabriel Dimitriu. “Optimal control for Lotka-Volterra systems with a hunter population”. In: *International Conference on Large-Scale Scientific Computing*. Springer. 2007, pp. 277–284.